

目录

SQL 注入漏洞修复报告	2
目录	2
1. SQL 注入漏洞概述	2
2. 漏洞一：注册功能 SQL 注入	2
2.1 问题代码	2
2.2 攻击示例	3
2.3 危害等级	3
3. 漏洞二：搜索功能 SQL 注入	3
3.1 问题代码	3
3.2 攻击示例	4
3.3 危害等级	4
4. 漏洞三：登录功能 SQL 注入	4
4.1 问题代码	4
4.2 攻击示例	4
4.3 危害等级	5
5. 漏洞四：首页用户查询 SQL 注入	5
5.1 问题代码	5
5.2 攻击分析	5
6. SQL 注入原理与危害	5
6.1 什么是 SQL 注入	5
6.2 原理示意图	5
6.3 SQL 注入的常见危害	6
7. 参数化查询详解	6
7.1 什么是参数化查询	6
7.2 实现方式	6
7.3 为什么参数化查询能防注入	6
7.4 其他防御方式的局限性	7
8. 修复方案	7
8.1 修复对照表	7
8.2 修复后核心代码	8
9. 修复前后代码对比	9
注册功能	9
搜索功能	9
登录功能	10
10. 安全建议	10
10.1 开发规范	10
10.2 SQL 注入防御金字塔	10
10.3 开发者检查清单	10
附录：OWASP SQL 注入预防指南	11

SQL 注入漏洞修复报告

项目名称：用户信息管理系统

报告日期：2026-07-08

审计目标：Flask 用户管理平台中的 SQL 注入漏洞分析与修复

报告人：安全审计组

目录

1. SQL 注入漏洞概述
2. 漏洞一：注册功能 SQL 注入
3. 漏洞二：搜索功能 SQL 注入
4. 漏洞三：登录功能 SQL 注入
5. 漏洞四：首页用户查询 SQL 注入
6. SQL 注入原理与危害
7. 参数化查询详解
8. 修复方案
9. 修复前后代码对比
10. 安全建议

1. SQL 注入漏洞概述

在本次安全审计中，发现目标系统存在 **4 处 SQL 注入漏洞**，全部由 **f-string 字符串拼接 SQL 语句** 导致。

编号	漏洞位置	风险等级	攻击入口
SQL-001	注册功能 /register	严重	用户名、密码、邮箱、手机号
SQL-002	搜索功能 /search	严重	keyword 参数
SQL-003	登录功能 /login	高危	用户名参数
SQL-004	首页 /	高危	session 中的用户名

2. 漏洞一：注册功能 SQL 注入

2.1 问题代码

```
@app.route("/register", methods=["GET", "POST"])
def register():
```

```

username = request.form.get("username", "")
password = request.form.get("password", "")
email = request.form.get("email", "").strip()
phone = request.form.get("phone", "").strip()

# 使用 f-string 字符串拼接 SQL ——存在 SQL 注入!
sql = f"INSERT INTO users (username, password, email, phone) VALUES ('{username}', '
cursor.execute(sql)

```

2.2 攻击示例

攻击者在用户名输入框中输入以下内容：

用户名：admin', 'hacked'), ('test
密码：任意

拼接后的 SQL 语句变为：

```
INSERT INTO users (username, password, email, phone) VALUES ('admin', 'hacked'), ('test
```

可一次性插入多条数据，甚至通过 '); DROP TABLE users -- 删除整个用户表。

2.3 危害等级

攻击类型	可能性	影响
数据篡改	高	任意修改数据库内容
数据删除	高	删除整个 users 表
数据窃取	中	通过报错信息获取数据结构

3. 漏洞二：搜索功能 SQL 注入

3.1 问题代码

```

@app.route("/search")
def search():
    keyword = request.args.get("keyword", "")

    # 使用 f-string 字符串拼接 SQL ——存在 SQL 注入!
    sql = f"SELECT * FROM users WHERE username LIKE '{keyword}%' OR email LIKE '{keywo
    print(f"[SQL] {sql}") # 后台打印 SQL 语句

```

```
cursor.execute(sql)
```

3.2 攻击示例

攻击者在搜索框输入：

```
' OR 1=1 --
```

拼接后的 SQL：

```
SELECT * FROM users WHERE username LIKE '%' OR 1=1 -- '%' OR email LIKE '%' OR 1=1 -- %
```

返回全部用户的敏感信息（用户名、邮箱、手机号）。

更严重的攻击：

```
' UNION SELECT id, username, password, phone FROM users --
```

可获取所有用户的密码明文。

3.3 危害等级

攻击类型	可能性	影响
数据泄露	极高	获取全部用户数据
拖库攻击	高	通过 UNION 查询获取密码
权限提升	中	结合其他漏洞提权

4. 漏洞三：登录功能 SQL 注入

4.1 问题代码

```
@app.route("/login", methods=["GET", "POST"])
def login():
    username = request.form.get("username", "").strip()

    # 使用 f-string 拼接 SQL —— 存在 SQL 注入！
    cursor.execute(f"SELECT * FROM users WHERE username='{username}'")
```

4.2 攻击示例

攻击者在用户名输入：

```
admin' --
```

拼接后的 SQL:

```
SELECT * FROM users WHERE username='admin' --'
```

-- 注释掉了后面的内容，无需密码即可登录 admin 账号。

4.3 危害等级

攻击类型	可能性	影响
未授权登录 权限提升	极高 高	任意用户登录 登录管理员账号

5. 漏洞四：首页用户查询 SQL 注入

5.1 问题代码

```
@app.route("/")
def index():
    username = session.get("username")

    # 使用 f-string 拼接 SQL —— 存在 SQL 注入!
    cursor.execute(f"SELECT * FROM users WHERE username='{username}'")
```

5.2 攻击分析

虽然 username 来自 session 而非用户直接输入，但如果攻击者通过其他漏洞（如 XSS）或 session 伪造（弱密钥 dev-key-2025）控制了 session 内容，同样可以触发 SQL 注入。

6. SQL 注入原理与危害

6.1 什么是 SQL 注入

SQL 注入是攻击者通过在输入字段中插入恶意的 SQL 代码，改变应用程序原本 SQL 语句的语义，从而实现对数据库的非法操作。

6.2 原理示意图

正常输入：admin

SQL: SELECT * FROM users WHERE username = 'admin'
作为数据处理

恶意输入： admin' OR 1=1 --
SQL：SELECT * FROM users WHERE username = 'admin' OR 1=1 --'
 闭合引号 → 变成代码！

用户输入的内容本应作为**数据**，但因为直接拼接到 SQL 中，变成了**代码**被执行。

6.3 SQL 注入的常见危害

危害类型	说明
未授权访问	绕过登录验证
数据泄露	窃取数据库中的敏感信息
数据篡改	插入、修改、删除数据
数据销毁	删除表甚至整个数据库
命令执行	部分数据库可执行系统命令
提权攻击	获取更高权限的账号

7. 参数化查询详解

7.1 什么是参数化查询

参数化查询（Parameterized Query）是将 SQL 语句与数据分离的编程方式。SQL 语句的**结构**由开发者定义，用户的输入仅作为**参数**传递给数据库引擎，永远不会被当作 SQL 代码执行。

7.2 实现方式

```
# SQL 注入漏洞版 (f-string 拼接)
sql = f"SELECT * FROM users WHERE username = '{username}'"
cursor.execute(sql)

# SQL 注入修复版 (参数化查询)
sql = "SELECT * FROM users WHERE username = ?"
cursor.execute(sql, (username,))
```

关键区别：- ? 是**占位符**，表示此处是一个参数 - (username,) 是**参数值**，数据库引擎会将其安全地绑定到占位符 - 用户的输入永远是**数据**，不会被解析为 SQL 代码

7.3 为什么参数化查询能防注入

攻击者输入： admin' OR 1=1 --

f-string 拼接:

```
"SELECT * FROM users WHERE username = 'admin' OR 1=1 --"
```

输入中的单引号闭合了SQL中的引号 → 注入成功

参数化查询:

```
"SELECT * FROM users WHERE username = ?" → 参数: "admin' OR 1=1 --"
```

数据库引擎将整个输入当作一个字符串值, 不会解析其中的 SQL 关键字

实际执行的查询等价于: WHERE username = "admin' OR 1=1 --"

去数据库里找用户名是 "admin' OR 1=1 --" 的用户 → 找不到 → 安全!

7.4 其他防御方式的局限性

防御方式	局限性	结论
过滤特殊字符	总有绕过方式 (编码、大小写、注释等)	不可靠
转义输入	不同数据库转义规则不同, 容易遗漏	不可靠
WAF / 防火墙	可以被绕过 (如你刚才在 SQL-Labs 的尝试)	不可靠
参数化查询	从设计上杜绝 SQL 注入	推荐

8. 修复方案

8.1 修复对照表

漏洞编号	漏洞位置	修复前 (f-string)	修复后 (参数化查询)
SQL-001	注册	f"INSERT INTO users VALUES ('{username}', ...)"	"INSERT INTO users VALUES (?, ?, ?, ?)"
SQL-002	搜索	f"SELECT * FROM users WHERE LIKE '%{keyword}%"	"SELECT * FROM users WHERE LIKE ?"
SQL-003	登录	f"SELECT * FROM users WHERE username='{username}'"	"SELECT * FROM users WHERE username = ?"
SQL-004	首页	f"SELECT * FROM users WHERE username='{username}'"	"SELECT * FROM users WHERE username = ?"

8.2 修复后核心代码

```
import sqlite3
from werkzeug.security import generate_password_hash, check_password_hash

# ===== 注册（参数化查询） =====
@app.route("/register", methods=["GET", "POST"])
def register():
    username = request.form.get("username", "").strip()
    password = request.form.get("password", "").strip()
    email = request.form.get("email", "").strip()
    phone = request.form.get("phone", "").strip()

    conn = sqlite3.connect("data/users.db")
    cursor = conn.cursor()
    try:
        # 【已修复】使用 ? 占位符，用户的输入只作为参数，不会变成 SQL 代码
        cursor.execute(
            "INSERT INTO users (username, password, email, phone) VALUES (?, ?, ?, ?)",
            (username, generate_password_hash(password), email, phone),
        )
        conn.commit()
    finally:
        conn.close()

# ===== 搜索（参数化查询） =====
@app.route("/search")
def search():
    keyword = request.args.get("keyword", "")

    conn = sqlite3.connect("data/users.db")
    cursor = conn.cursor()
    # 【已修复】使用 ? 占位符，keyword 中的恶意内容不会被解析为 SQL
    cursor.execute(
        "SELECT * FROM users WHERE username LIKE ? OR email LIKE ?",
        (f"%{keyword}%", f"%{keyword}%"),
    )
    rows = cursor.fetchall()
    conn.close()

# ===== 登录（参数化查询） =====
@app.route("/login", methods=["GET", "POST"])
```



```
def login():
    username = request.form.get("username", "").strip()

    conn = sqlite3.connect("data/users.db")
    cursor = conn.cursor()
    # 【已修复】使用 ? 占位符
    cursor.execute("SELECT * FROM users WHERE username = ?", (username,))
    row = cursor.fetchone()
    conn.close()

    if row and check_password_hash(row["password"], password):
        # 登录成功
```

9. 修复前后代码对比

注册功能

```
# ===== 漏洞版 (f-string 拼接) =====
sql = f"INSERT INTO users VALUES ('{username}', '{password}', '{email}', '{phone}')"
cursor.execute(sql)

# ===== 修复版 (参数化查询) =====
cursor.execute(
    "INSERT INTO users (username, password, email, phone) VALUES (?, ?, ?, ?)",
    (username, generate_password_hash(password), email, phone),
)
```

搜索功能

```
# ===== 漏洞版 (f-string 拼接) =====
sql = f"SELECT * FROM users WHERE username LIKE '%{keyword}%' OR email LIKE '%{keyword}%'"
print(f"[SQL] {sql}") # 特意打印 SQL 方便观察注入
cursor.execute(sql)

# ===== 修复版 (参数化查询) =====
cursor.execute(
    "SELECT * FROM users WHERE username LIKE ? OR email LIKE ?",
    (f"%{keyword}%", f"%{keyword}%"),
)
```

登录功能

```
# ===== 漏洞版 (f-string 拼接) =====
cursor.execute(f"SELECT * FROM users WHERE username='{username}'")

# ===== 修复版 (参数化查询) =====
cursor.execute("SELECT * FROM users WHERE username = ?", (username,))
```

10. 安全建议

10.1 开发规范

规范	说明
永远使用参数化查询	任何 SQL 语句都必须使用？占位符
禁止字符串拼接 SQL	同样禁止在存储过程、ORM 原生查询中拼接
最小权限原则	数据库连接使用只读账号，不用 root
输入验证	对输入做长度、格式校验作为辅助防御
错误信息处理	不要将数据库错误信息直接返回给用户

10.2 SQL 注入防御金字塔

参数化查询	← 核心防线，杜绝注入
输入验证	← 辅助防御，限制输入格式
最小权限	← 降低危害，限制数据库权限
WAF / 防火墙	← 外部防御，最后一道屏障

10.3 开发者检查清单

- ☐ 所有 SQL 语句都使用参数化查询
- ☐ 没有使用 f-string / % 格式化拼接 SQL
- ☐ 数据库连接使用最小权限账号
- ☐ 错误信息不暴露 SQL 语句或数据库结构
- ☐ 不将 SQL 日志输出到前端或公开日志
- ☐ 定期进行代码安全审计

附录：OWASP SQL 注入预防指南

OWASP（开放式 Web 应用程序安全项目）关于 SQL 注入防御的建议：

1. **首选防御：参数化查询（Prepared Statements）**
 - 所有 SQL 语句必须使用参数化接口
 - 用户输入只能作为参数传递，不能拼接到 SQL 模板中
2. **存储过程：也使用参数化调用**
 - 存储过程内部同样不应拼接 SQL
3. **转义是最后手段，不是替代方案**
 - 特殊字符过滤和转义只能作为深度防御，不能替代参数化查询
4. **最小数据库权限**
 - 应用程序使用仅具有必要权限的数据库账户
 - 查询只读操作使用只读账号

报告编写： Claude Code AI Assistant

报告版本： v1.0

参考标准： OWASP Top 10 (2021) - A03:2021 Injection

测试环境： SQL-Labs 靶场 + Flask 应用

免责声明： 本报告仅用于安全学习和教学目的